

TRABALHO FINAL – parte 3: implementação do analisador sintático

Implementar o **analisador sintático** de forma que indique quais programas escritos na linguagem 2026.1 estão sintaticamente corretos. Deve-se implementar também **tratamento de erros** sintáticos, quais sejam: *tokens* que não estão na sequência definida na gramática. Se necessário, efetue as correções/alterações nas especificações dos *tokens*, conforme indicado na **avaliação do analisador léxico**, e da gramática, conforme indicado na **avaliação do trabalho no.3**.

O analisador sintático (para Windows 8 ou superior) deve ser implementado em Java, conforme especificado abaixo. Na implementação do analisador sintático deve ser utilizado o GALS, ferramenta para geração de analisadores léxicos e sintáticos.

Entrada	– A entrada para o analisador sintático é um conjunto de caracteres, isto é, o programa fonte do editor do compilador.
Saída	– Caso o botão compilar seja pressionado (ou a tecla de atalho correspondente), a ação deve ser: executar as análises léxica e sintática do programa fonte e apresentar a saída. Um programa pode ser compilado com sucesso ou apresentar erros. Em cada uma das situações a saída deve ser: <u>1ª situação</u>: programa compilado com sucesso ✓ apagar o conteúdo da área reservada para mensagens; ✓ apresentar a mensagem (<i>programa compilado com sucesso</i>), na área reservada para mensagens, indicando que o programa não apresenta erros. A lista de tokens <u>não deve mais</u> ser mostrada na área reservada para mensagens. <u>2ª situação</u>: programa apresenta erros ✓ apagar o conteúdo da área reservada para mensagens; ✓ apresentar a mensagem, na área reservada para mensagens, indicando que o programa apresenta erro. O erro pode ser léxico ou sintático, cujas mensagens devem ser conforme relacionado abaixo: • para erro léxico: indicar a <u>linha</u> onde ocorreu o erro e a <u>descrição</u> do erro, emitindo uma mensagem adequada, conforme especificado na parte 2 do trabalho final; • para erro sintático: indicar a <u>linha</u> onde ocorreu o erro, o <u>token encontrado</u> (lexema) e o(s) <u>símbolo(s) esperado(s)</u>, conforme explicado em aula e detalhado a seguir. As mensagens geradas pelo GALS devem ser alteradas conforme as orientações dadas em aula.

OBSERVAÇÕES:

- O tipo do analisador sintático a ser gerado é **LL (1)**.
- As **mensagens de erro** devem ser conforme exemplos abaixo:
linha 1: encontrado EOF esperado expressão
linha 1: encontrado area esperado (

Observa-se que:

- quando for encontrado uma constante_string, como por exemplo "olá mundo", a mensagem deve ser: encontrado constante_string esperado ...¹
- quando for encontrado \$, a mensagem deve ser: encontrado EOF esperado ...
- quando for encontrado símbolo especial, palavra reservada, identificador, constante_int, constante_float ou constante_char, deve ser apresentado o lexema na mensagem de erro, como, por exemplo, em: encontrado ; esperado ...; encontrado main esperado ...; encontrado area esperado ...; encontrado 123 esperado ...; encontrado 12.3 esperado ...; encontrado \n esperado ...
- quando for esperado fim de programa, a mensagem deve ser: encontrado ... esperado EOF
- quando for esperado identificador, constante_int, constante_float, constante_char ou constante_string, a mensagem deve ser, respectivamente: encontrado ... esperado identificador, encontrado ... esperado constante_int, encontrado ... esperado constante_float, encontrado ... esperado constante_string
- as palavras reservadas e os símbolos especiais devem ser escritos nas mensagens de erro tal como definido na linguagem, ou seja, exatamente como o programador deve escrevê-los nos programas
- para o não-terminal <lista_de_expressoes>, ou com outro nome, usado para definir essa estrutura sintática especificada no trabalho no.3, a mensagem deve ser: encontrado ... esperado expressao
- para os não-terminais <expressao>, <expressao_>, <termo_logico>, <relacional>, <relacional_>, <aritmetica>, <aritmetica_>, <termo_aritmetico>, <termo_aritmetico_>, <fator>, <fator_>, <elemento>, a mensagem deve ser: encontrado ... esperado expressao
- para os demais não-terminais, a mensagem deve ser: encontrado ... esperado simbolo₀₁, simbolo₀₂, ... simbolo_{0n}, conforme tabela de análise sintática (exemplificado a seguir);
- todas as mensagens de erro geradas pelo GALS devem ser mantidas (em **comentário de linha** ao lado da mensagem alterada - arquivo ParserConstants.java), MAS devem ser alteradas, seguindo as orientações dadas em aula.

¹ Em todo o enunciado do trabalho ... representa o(s) símbolo(s) esperados.

Por exemplo, considerando o seguinte “trecho” da tabela de análise sintática (no GALS, menu Documentação > Tabela de Análise Sintática):

	id	pr_ask	pr_bool	pr_char	pr_define	pr_float	pr_if	pr_int	pr_main	pr_repeat	pr_string	pr_tell	"_"	"<_"
<programa>	-	-	-	-	-	-	-	-	0	-	-	-	-	-
<lista_instrucoes>	1	1	-	-	-	-	1	-	-	1	-	1	-	-
<declaracao_var>	3	3	-	-	2	-	3	-	-	3	-	3	-	-
<lista_id>	4	-	-	-	-	-	-	-	-	-	-	-	-	-
<tipo>	-	-	5	6	-	7	-	8	-	-	9	-	-	-
<atribuicao_>	-	-	-	-	-	-	-	-	-	-	-	-	22	23

As mensagens de erro para os não-terminais relacionados devem ser:

- para o não-terminal <programa>: **encontrado ... esperado** main
- para o não-terminal <lista_instrucoes>: **encontrado ... esperado** identificador ask if repeat tell
- para o não-terminal <declaracao_var>: **encontrado ... esperado** identificador ask define if repeat tell
- para o não-terminal <lista_id>: **encontrado ... esperado** identificador
- para o não-terminal <tipo>: **encontrado ... esperado** bool char float int string
- para o não-terminal <atribuicao_>: **encontrado ... esperado** = <-
- para o não-terminal <expressao>: **encontrado ... esperado** expressao

- As especificações feitas no trabalho nº3 (e já corrigidas) devem usadas para implementação do analisador sintático. Observa-se que: (1) todas as alterações/correções indicadas na avaliação do analisador léxico e na avaliação do trabalho nº3 devem ser efetuadas, caso contrário, o analisador sintático receberá nota 0.0 (zero); (2) o analisador sintático desenvolvido usando especificações diferentes daquelas elaboradas pela equipe no trabalho nº3 (com as devidas correções, se for o caso) receberá nota 0.0 (zero).
- A implementação do analisador sintático, bem como da interface do compilador e do analisador léxico, deve ser disponibilizada no **AVA**, na aba **COMPILADOR**, na tarefa “**parte 3 - analisador sintático**”. Deve ser disponibilizado um **arquivo compactado** (com o nome: `sintatico`, seguido do número da equipe), contendo o projeto completo, incluindo **código fonte**, **arquivo .jar**, **arquivo com as especificações léxica e sintática** (arquivo `.gals`) e demais recursos. Basta um integrante da equipe postar o trabalho. **Caso não sejam postados todos os arquivos solicitados ou não seja possível rodar o arquivo .jar, será atribuída nota 0.0 (zero) ao analisador sintático.**
- Na avaliação do analisador sintático serão levadas em consideração: a correta especificação do arquivo `.gals`, conforme as correções/alterações do trabalho nº3; a qualidade das mensagens de erro, conforme descrito acima. Observa-se que todas as mensagens de erro sintático geradas pelo GALS devem ser alteradas conforme especificado anteriormente e mantidas em comentário de linha ao lado da mensagem alterada (arquivo `ParserConstants.java`), só podendo ser “informado o símbolo esperado” a partir desse código, caso contrário, **será atribuída nota 0.0 (zero) ao analisador sintático.**

DATA: entregar o trabalho até às 23h do dia 16/05/2026 (sábado). **Só serão aceitas entregas via AVA até data e hora determinadas.**

EXEMPLOS DE ENTRADA / SAÍDA

EXEMPLO 1: com erro léxico

ENTRADA		SAÍDA (na área de mensagens)
linha	1: main 2: define 3: int : lado; 4: 5: ask ("digite um valor para lado: ", lado 6: tell ("o valor digitado foi: ", lado); 7: end	linha 5: constante_string inválida

EXEMPLO 2: com erro sintático

ENTRADA		SAÍDA (na área de mensagens)
linha	1: main 2: define 3: int : lado; 4: 5: ask ("digite um valor para lado: ", lado 6: tell ("o valor digitado foi: ", lado); 7: end	linha 6: encontrado tell esperado ,)

EXEMPLO 3: sem erro

ENTRADA		SAÍDA (na área de mensagens)
linha	1: main 2: define 3: int : lado; 4: 5: ask ("digite um valor para lado: ", lado); 6: tell ("o valor digitado foi: ", lado); 7: end	programa compilado com sucesso